

torch.tensor#

<https://pytorch.org/docs/stable/generated/torch.tensor.html>

Description:

Constructs a tensor with no autograd history (also known as a “leaf tensor”, see Autograd mechanics) by copying data. When working with tensors prefer using `torch.Tensor.clone()`, `torch.Tensor.detach()`, and `torch.Tensor.requires_grad_()` for readability. Letting `t` be a tensor, `torch.tensor(t)` is equivalent to `t.detach().clone()`, and `torch.tensor(t, requires_grad=True)` is equivalent to `t.detach().clone().requires_grad_(True)`. `torch.as_tensor()` preserves autograd history and avoids copies where possible. `torch.from_numpy()` creates a tensor that shares storage with a NumPy array.

Function Signature

```
torch.tensor(data, *, dtype=None, device=None,
requires_grad=False, pin_memory=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if `None`, infers data type from data. `device` (`torch.device`, optional) – the device of the constructed tensor. If `None` and data is a tensor then the device of data is used. If `None` and data is not a tensor then the result tensor is constructed on the current device. `requires_grad` (`bool`, optional) – If autograd should record operations on the returned tensor. Default: `False`. `pin_memory` (`bool`, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: `False`.

Code Examples

```
>>> torch.tensor([[0.1, 1.2], [2.2, 3.1], [4.9, 5.2]])
tensor([[ 0.1000,  1.2000],
        [ 2.2000,  3.1000],
        [ 4.9000,  5.2000]])

>>> torch.tensor([0, 1]) # Type inference on data
tensor([ 0,  1])

>>> torch.tensor([0.11111, 0.22222, 0.3333333],
...               dtype=torch.float64,
...               device=torch.device('cuda:0')) # creates a
double tensor on a CUDA device
tensor([ 0.1111,  0.2222,  0.3333], dtype=torch.float64,
device='cuda:0')

>>> torch.tensor(3.14159) # Create a zero-dimensional (scalar)
tensor
tensor(3.1416)

>>> torch.tensor([]) # Create an empty tensor (of size (0,))
tensor([])
```

Notes & Warnings

WARNING

When working with tensors prefer using `torch.Tensor.clone()`, `torch.Tensor.detach()`, and `torch.Tensor.requires_grad_()` for readability. Letting `t` be a tensor, `torch.tensor(t)` is equivalent to `t.detach().clone()`, and `torch.tensor(t, requires_grad=True)` is equivalent to `t.detach().clone().requires_grad_(True)`.

NOTE

See also `torch.as_tensor()` preserves autograd history and avoids copies where possible. `torch.from_numpy()` creates a tensor that shares storage with a NumPy array.

Detailed Documentation

Documentation

Constructs a tensor with no autograd history (also known as a “leaf tensor”, see Autograd mechanics) by copying data.

When working with tensors prefer using `torch.Tensor.clone()`, `torch.Tensor.detach()`, and `torch.Tensor.requires_grad_()` for readability. Letting `t` be a tensor, `torch.tensor(t)` is equivalent to `t.detach().clone()`, and `torch.tensor(t, requires_grad=True)` is equivalent to `t.detach().clone().requires_grad_(True)`.

`torch.as_tensor()` preserves autograd history and avoids copies where possible. `torch.from_numpy()` creates a tensor that shares storage with a NumPy array.

`data (array_like)` – Initial data for the tensor. Can be a list, tuple, NumPy ndarray, scalar, and other types.

`dtype (torch.dtype, optional)` – the desired data type of returned tensor. Default: if None, infers data type from data.

`device (torch.device, optional)` – the device of the constructed tensor. If None and data is a tensor then the device of data is used. If None and data is not a tensor then the result tensor is constructed on the current device.

`requires_grad (bool, optional)` – If autograd should record operations on the returned tensor. Default: False.

`pin_memory (bool, optional)` – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

